ERGALICS STUDIO · 技术文档

电磁谐振特征值求解器

让每一次科学计算，都发生在离思想最近的地方

面向微波器件、天线与电磁兼容的稀疏厄密非正定本征问题。纯 Python 与 NumPy 实现，内置厚重启 Lanczos、块 LOBPCG、Jacobi-Davidson 三种内核，按真实残差认证收敛。

官网
snowleopard-io.github.io/ErgalicsStudio

GitHub
github.com/SnowLeopard-io/ErgalicsStudio

Gitee
gitee.com/cnt-code/ergalics-studio

本篇章节

一 问题定义与适用边界
二 求解内核
三 位移策略与自适应位移
四 收敛性控制与精度认证
五 内存与复杂度
六 并行与运行环境
七 插件运行时架构
八 界面与交互
九 内置样例与基准数据
十 故障场景与处置建议
十一 测试与可复现性
十二 安全与合规设计
十三 与赛题要求的对应
十四 已知边界与后续工作

独立专题文档 · Standalone Note

目 录

一、问题定义与适用边界 04

二、求解内核 05

三、位移策略与自适应位移 09

四、收敛性控制与精度认证 10

五、内存与复杂度 11

六、并行与运行环境 12

七、插件运行时架构 14

八、界面与交互 16

九、内置样例与基准数据 24

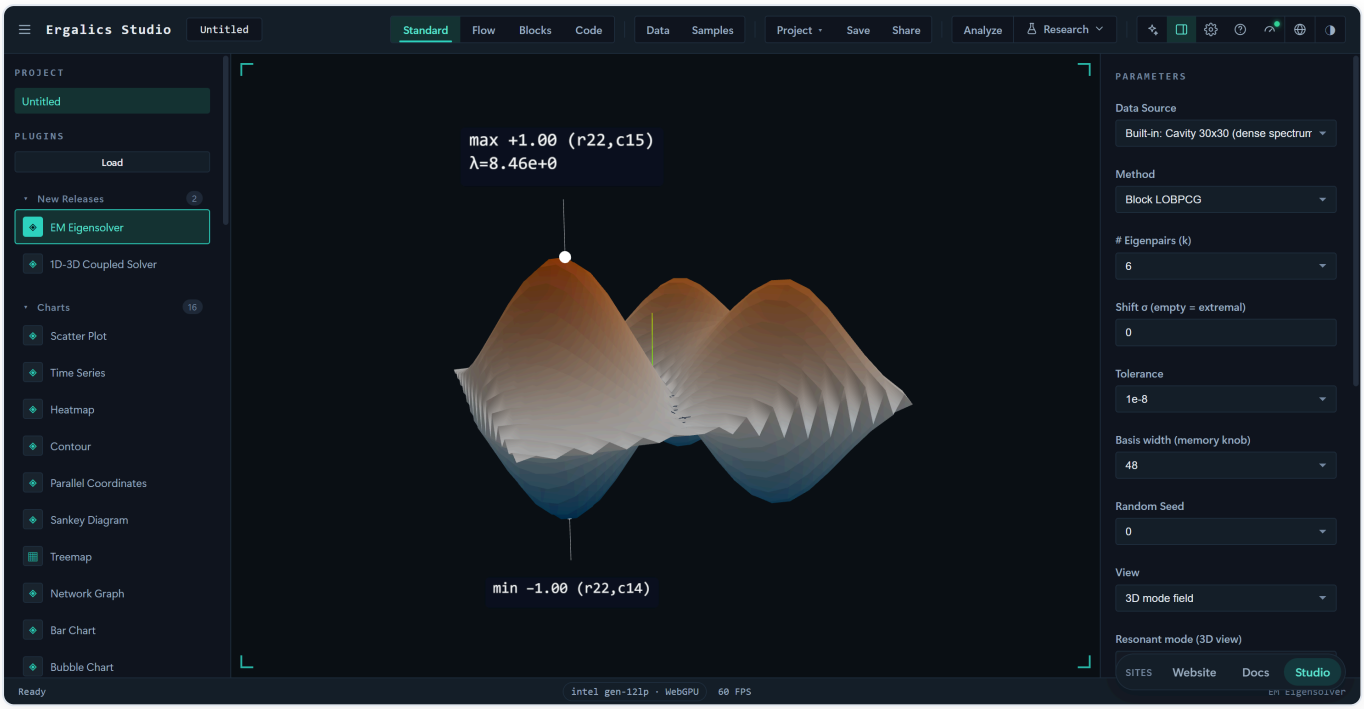
十、故障场景与处置建议 32

十一、测试与可复现性 33

十二、安全与合规设计 41

十三、与赛题要求的对应 41

十四、已知边界与后续工作 43



`cavity_small`（30 × 30）配合块 LOBPCG 内核求解后的 3D 模式场：λ = 8.42e+0 模式的空间位移曲面，发散色标按 Okabe-Ito 配色取正峰朱红、负谷蓝，与 8.3 节一致的渲染路径。

电磁谐振特征值求解器（EM Eigensolver）是一个内置插件（`src/plugins/builtin/em-eigensolver/`，注册 id `example.em-eigensolver`，分类"物理模拟 physics"，沙箱级别 `trusted`），用于求解微波器件、天线、电磁兼容与光子晶体等场景中由有限元（FEM）或矩量法（MoM）离散得到的**大规模稀疏厄密非正定矩阵特征值问题** `A x = λ x`。插件在宿主侧提供参数面板与求解报告，在独立的 Web Worker 中用 Pyodide 运行配套的纯 Python 包 `em_eigensolver`，并把谱、真实残差、外迭代收敛轨迹与谐振模式场渲染到画布或 Three.js 场景中。

求解器本身不依赖任何闭源组件：核心算法是纯 Python + NumPy 实现，SciPy 存在时只用于更成熟的稀疏存储与向量内核；整个流程从不构造或稠密化 `n × n` 算子，唯一的稠密分解作用在规模不超过 `basis_dim` 的投影矩阵上。

一 问题定义与适用边界

1.1 从麦克斯韦方程到厄密本征问题

电磁谐振分析在离散化之后归结为稀疏厄密矩阵的本征问题。本插件处理的是**标准形式** `A x = λ x`：`A` 为 `n × n` 稀疏厄密矩阵（实对称或复厄密），`n` 可到十万量级。材料色散、复杂边界条件（如 PML）与高频/低频效应使 `A` 不再正定，目标频率对应的位移 `σ` 可能落在谱内部，此时 `A − σI` 可以接近奇异——这是本插件与常规"只处理正定矩阵"工具的差别所在。

1.2 三类困难场景

场景	数学表现	朴素做法的失败方式
零附近密集谱（低频击穿）	大量特征值挤在 `σ` 附近甚至 `σ = 0`	位移逆变换的内层线性系统病态，迭代次数爆炸或结果错误
重特征值	同一特征值有多重（简并模式）	单向量迭代收敛到同一个向量，其余简并模式丢失
近奇异位移	`σ` 与某个特征值几乎重合	`(A − σI)⁻¹` 元素巨大，残差被放大，收敛判据失真

这三类场景在内置样例中都有对应：`cluster_zero` 在零附近给出 720 阶中的 153 个 `abs(λ) < 0.05` 的特征值，`degenerate_pair` 的每个特征值都至少二重，两者默认位移恰好落在谱内部。插件针对它们分别提供了块迭代与收缩（deflation）机制、自适应位移策略，以及按真实残差认证的收敛判据。

1.3 适用边界

维度	支持范围	说明
矩阵类型	实数对称或复厄密（`A = Aᴴ`）	非厄密、非对称矩阵不在求解范围内
问题形式	标准本征问题 `A x = λ x`	广义问题 `A x = λB x` 未实现，需使用者先用 `B` 的因子变换

维度	支持范围	说明
规模	`n` 从个位数到 `1e5` 量级	内置 `cavity_large` 样例 `n = 102400`，`nnz = 510720`
谱位置	极端特征值或谱内目标位移	位移由 `σ` 指定，支持自适应偏移
存储	`.mtx`（Matrix Market）、`.npz`（SciPy 稀疏或稠密归档）、`.npy`（稠密，受尺寸护栏）；本地 CPython 另支持 `.h5`/`.hdf5`（h5py）、`.fits`（astropy）、`.nc`（SciPy NetCDF）可选直读	稠密输入即刻稀疏化，`n² ≤ 4×10⁶` 才接受；工程格式缺包时抛出指名缺失包的 `MatrixIOError`
运算环境	CPython + NumPy（SciPy 可选）；浏览器 Pyodide	CPython 用 SciPy 稀疏存储与向量内核；纯 NumPy 后端自带行块多线程 matvec（见第六节）；WebGPU SpMV 内核为平台级资产

二 求解内核

三个迭代内核加上一条稠密直解兜底，统一由 `SolverConfig` 与 `solve()` 门面（`python/em_eigensolver/solver.py`）路由。

内核	实现文件	递推与投影空间	主要适用场景
厚重启 Lanczos	`lanczos.py`	三对角 Krylov 子空间，厚重启保留 `k` 个 Ritz 向量加最后一个 Lanczos 向量	极端特征值；目标落在谱间隙时的位移逆变换
块 LOBPCG	`lobpcg.py`	`span{X, R, P}` 三块 Rayleigh-Ritz（块宽 `B`）	极端特征值，且存在聚簇或重特征值
Jacobi-Davidson	`jacdavid.py`	搜索空间 `V` 加投影校正方程，带收缩锁定	谱内密集簇中的目标、近奇异位移
稠密直解	`solver.py`	`numpy.linalg.eigh`	`n ≤ dense_threshold`（默认 800）的小问题

2.1 方法路由

`method` 取 `auto`（默认）时按位移是否存在分流；显式指定内核则直接调用。`which` 在无位移时决定取哪一端（`LM` 最大幅值、`LA` 最大代数、`SA` 最小代数）。

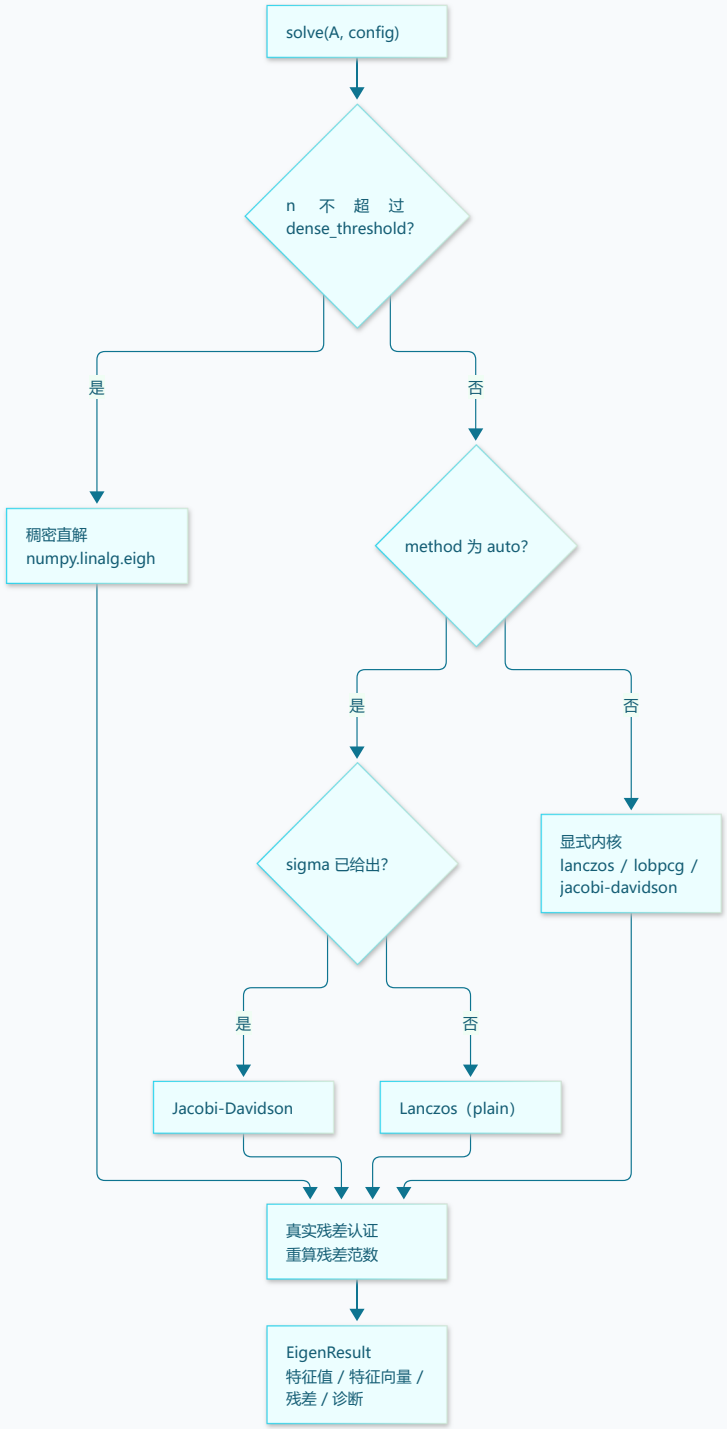


图 1

插件界面上的算法下拉把这条路由显式暴露给使用者（"自动（有 σ → JD，无 σ → Lanczos）"），并允许强制指定三个内核之一。需要说明的是，插件固定以 `which = 'LM'` 发起请求，界面没有提供极端类型选择；`which` 的完整取值只在 Python 包与 CLI 层可用。

2.2 厚重启 Lanczos（Krylov-Schur）

`lanczos_solve()` 每轮把 Krylov 基扩张到宽度 `m = max(basis_dim, k + 8)`（上限 `n`），在 `m × m` 的三对角投影上做 Rayleigh-Ritz，然后按 Wu & Simon 的厚重启策略重建基：保留 `k` 个目标 Ritz 向量与最后一个 Lanczos 向量，把已收敛信息全部带过去，而不是丢弃整段 Krylov 空间重新开始。投影矩阵强制对称化（`H = (H + Hᴴ)/2`）以抵消浮点累积的非厄密性。

- **正交化**：每步做两遍 DGKS 全重正交（`for pas in (0, 1)`），投影系数在两遍中累加进 `H`；基础向量范数低于 `1e-14` 时判为到达不变子空间，改用随机方向继续扩张并把该列次对角元置 0。
- **两种谱变换**：`sigma` 为空时逐次计算 `A q`（plain，求极端特征值）；给出 `sigma` 时逐次调用 MINRES 解 `(A − σI)⁻¹ q`（位移逆），Ritz 值按 `λ = σ + 1/θ` 映射回原谱。
- **收敛判据**：每个周期都用真实 matvec 重算 `‖A y − λ y‖`，除以该次返回的最大 `|λ|`，全部小于容差才算收敛。
- **诊断输出**：周期数、基宽、当前位移、残差估计上界（末行耦合 `|lastrow · S|` 的最大值）均写入 `diagnostics`。

2.3 MINRES 内层求解器

位移逆变换与 JD 的校正方程都需要解厄密**不定**线性系统，共轭梯度不再适用，因此内层统一使用 MINRES（`minres.py`）。

设计点	做法
极小化问题	Hermitian Lanczos 三项递推构造基，在 `(k+1) × k` 三对角投影上解最小二乘 `min ‖β₁e₁ − T_k y‖`
求解方式	每 `check_every = 4` 步调用一次 LAPACK `lstsq`；`T_k` 规模受 `restart` 限制（默认 80），开销可忽略
重启	基宽达 `restart` 或最小二乘残差达标即停，物化增量解并**用一次额外 matvec 重算真实残差**

设计点	做法
判据	真实残差判据，不使用递推式累积量；返回 `converged` / `maxiter` 状态
预条件	默认不施加。理由见下

内层**有意做不精确求解**：外层 Rayleigh-Ritz 重正交会恢复精度，而近奇异的 `A − σI` 只消耗有界的内层迭代次数。位移逆路径上外层有效容差相应放宽为 `max(tol, 20 × minres_rtol)`。反过来说，若使用者把容差设到 `1e-12` 而内层相对残差是 `1e-6`，外层实际能达到的精度上限就是 `2×10⁻⁵`——这是不精确位移逆的固有代价，也是收敛报告里残差与容差之间为什么会出现约两个数量级间隔的原因。

内层预条件被有意省略，两个方向都验证过代价：不定算子的 Jacobi 预条件 `M = diag(A)` 并不正定，负的 `M` 内积会产生 NaN；对称对角缩放 `A' = PAP`（`P = diag(1/√|d|)`）在对角元接近零时会把条件数改坏——实测某算例从干净收敛变为 400 次 MINRES 迭代仍未收敛。因此对对角结构良好的矩阵，文档建议的做法是先做显式对称变换再求解，而不是让内层自动施加预条件。块 LOBPCG 保留了 SPD 预条件钩子 `precond`，但门面 `solve()` 目前不传入任何预条件。

2.4 块 LOBPCG

`lobpcg_solve()` 实现经典三基 Rayleigh-Ritz：搜索空间取 `span{X, R, P}`，其中 `X` 是当前 Ritz 块、`R = AX − X·diag(θ)` 是残差块、`P` 是上一轮的方向块。块宽 `B` 由门面按 `max(basis_dim // 3, k + 4)` 计算，投影矩阵规模为 `3B × 3B`，稠密工作量保持在 `O(B³)`。

块形式是它处理重特征值的机制：同一个简并特征值的多个正交模式可以同时存在于 `X` 中，一轮迭代就能一起收敛，无需收缩逻辑。正交化采用带重正交的修正 Gram-Schmidt；遇到线性相关列时用随机方向替换，空间耗尽后**宁可让块变窄也不填充噪声方向**。收敛判据是尚未被选中的 `k` 个最小残差列全部达标；迭代结束后再从 `XᴴAX` 做一次 Rayleigh-Ritz 并重算 `k` 个真实残差。

需要注意的是，这个内核只求极端特征值：门面按 `which != 'SA'` 映射到"最大"或"最小"一侧，位移 `σ` 对它不生效。若在界面上选择 LOBPCG 同时填写了 `σ`，`σ` 会被忽略。

2.5 Jacobi-Davidson

`jd_solve()` 是应对谱内密集簇与近奇异位移的主力内核，迭代为 Sleijpen & van der Vorst 的标准流程：

1. 在当前搜索空间 `V` 上做 Rayleigh-Ritz（`H = VᴴAV`，对称化后 `eigh`），取距离 `σ` 最近的 Ritz 对；
2. 用显式的 `AV` 列计算精确残差 `r = A u − θ u`；
3. 近似求解校正方程 `(I − uuᴴ − WWᴴ)(A − θI)(I − uuᴴ − WWᴴ) t = −r'`，其中 `W` 是已锁定的特征向量基。两个斜投影把 `θ` 从谱中移开，因此即使 `θ ≈ λ`（近奇异位移）校正系统依然可解；内层仍由 MINRES 不精确求解，因为 JD 只需要一个下降方向；
4. 把正交化后的校正向量扩张进 `V`，基宽达到 `basis_dim` 时按"保留最好的 Ritz 向量、不足处补随机方向"的方式重启。

收缩（deflation）与简并模式：收敛的特征对被锁定进 `W` 并从后续搜索中投影剔除。判定某个 Ritz 对是否已锁定需要**同时**满足两个条件——特征值相对匹配（`|θ − λ_locked| ≤ 1e-9 · max(scale, |θ|)`）且 Ritz 向量与已锁定向量平行度大于 0.9。只用特征值判据会把重特征值的正交副本当作"已找到"而跳过，这正是简并模式丢失的原因。

有效内层预算：门面传 `minres_rtol = max(cfg.minres_rtol, 1e-4)`（内层容差恒不严于 `1e-4`——JD 只需要一个下降方向，内层过紧只会浪费迭代）与 `minres_maxiter = min(cfg.minres_maxiter, 160)`；按默认配置 `minres_rtol = 1e-6`，因此 JD 内层实际使用 `rtol = max(1e-6, 1e-4) = 1e-4`、`maxiter = min(250, 160) = 160`。对应测试 `test_jd_inner_tolerance_is_relaxed` 断言的就是这条 `max(·, 1e-4)` 收紧逻辑。

2.6 稠密直解路径

当 `n ≤ dense_threshold`（插件固定传 800）且矩阵对象提供 `todense()` 或 `toarray()` 时，`solve()` 直接走 `numpy.linalg.eigh`。这是一条**有界**路径，不构成"把稀疏矩阵显式转成稠密矩阵"的违规：触发条件是小到稠密特征分解比任何迭代路径都便宜，其 `O(n²)` 工作区（`n = 800` 时约 5 MB）比迭代法所需的 `n × basis_dim` 基块更小。该路径返回 `method = 'dense-lapack'`，`iterations` 与 `matvecs` 均为 0。

这一点对使用体验有直接影响：内置示例文件 `em-cavity-degenerate.mtx` 是 600 阶，低于 800 的阈值，因此从"内置样例"下拉里选它并点求解，走的就是稠密直解，报告中的算法字段会显示 `dense-lapack` 而不是三个迭代内核中的任何一个。

三 位移策略与自适应位移

位移 `σ` 是电磁谐振求解的核心输入：`σ` 对应目标频率，实际要找的是距 `σ` 最近的一批特征值。当 `σ` 数值上落在某个特征值上时，`A − σI` 近奇异，内层 MINRES 的迭代次数会急剧上升甚至无法在预算内收敛。

`lanczos_solve()` 在外层周期边界监测这一现象并自动偏移 `σ`：

触发条件	判定
内层迭代速率过高	最近 16 次内层求解中，达到 `near_singular_threshold = 120` 步的比例超过 30%
内层不收敛	任意一次 MINRES 返回未收敛

触发后按固定方向、步长倍增的方式移动：首次偏移取 `step = max(1e-3 × ‖A‖, 1e-2 × |σ|)`，方向为 `sign(σ)`（`σ = 0` 时取正），之后每次触发把步长乘 2；最多移动 `max_shift_moves = 5` 次，每次移动都追加进 `shift_history` 并整体重建 Krylov 基（变换后的算子已经不同，旧基不再有效）。内层计数窗口在移动后清零，`inner_converged` 标志复位。

两个被排除的替代方案有明确的实测理由：按比例取步长在 `σ ≈ 0` 时步长趋近于零、位移几乎不动；让方向在正负之间交替则会在谱附近来回振荡而不逃离奇异区。

自适应位移只作用于**位移逆 Lanczos**。Jacobi-Davidson 依靠校正方程的投影天然规避近奇异（`θ` 被投影移出谱），因此不需要偏移 `σ`；LOBPCG 不涉及位移。插件界面上"位移 σ（留空=极端特征值）"文本框中，留空表示不设位移、转而求极端特征值。

四 收敛性控制与精度认证

三个内核的收敛判据都基于**真实残差**，即用新的 matvec 重新计算 `‖A y − λ y‖`，而不用投影空间内的估计量。这一点在实现上是硬约束：Lanczos 的 `exact_residuals()` 每个周期为每个目标 Ritz 向量各做一次 matvec；LOBPCG 直接用本轮的 `AX` 计算 `R = AX − Xθ`；JD 在锁定后重新对每个输出向量做一次 matvec 校验。

内核	残差定义	相对量分母	收敛条件	迭代计数含义
Lanczos	`‖A y − λ y‖`	该次返回的最大 `|λ|`	全部 `k` 个残差 ≤ 有效容差	外层重启周期数
LOBPCG	`‖A x − θ x‖`	当前 Ritz 值绝对值的最大值	所选 `k` 列全部 ≤ 容差	迭代步数

内核	残差定义	相对量分母	收敛条件	迭代计数含义
Jacobi-Davidson	`‖A u − θ u‖`	`max(max|λ_locked|, 1e-3 × scale)`	每个被锁定对达标，锁定数 ≥ `k`	外层扩张步数
稠密直解	`‖A y − λ y‖`	全部特征值绝对值的最大值	全部残差 ≤ 容差	0（不适用）

分母取该次求解的**最大** `|λ|` 而不是逐特征值各自的分母，使小特征值的相对残差判据更严格、更保守。这一取舍意味着报告的残差略偏大，不会给出虚高的精度评价。

有效容差方面有两处需要使用者知道：位移逆 Lanczos 会把外层容差放宽到 `max(tol, 20 × minres_rtol)`（默认内层 `1e-6`，即下限 `2×10⁻⁵`）；LOBPCG 与 JD 直接用 `tol`。此外 `SolverConfig.normalized()` 会拒绝 `k < 1`、`tol ≤ 0`、未知 `method`，并给 `jacobi-davidson` 在未指定位移时补上 `σ = 0`（JD 必须有目标）。

每次求解返回的 `diagnostics` 字典包含：`k`、`n`、`tol`、`sigma`、`inner_iterations`（内层 MINRES 迭代总数）、`shift_history`（自适应位移轨迹）、`memory_hint_mb`（基块内存估算），以及各内核自身的字段（Lanczos 的 `cycles` / `basis` / `backend_shift` / `residual_estimate_max`，LOBPCG 的 `block`，JD 的 `locked` / `last_rel`）。

五 内存与复杂度

每轮迭代的开销集中在两处：一次 `O(nnz)` 的稀疏矩阵向量乘，一次对宽度 `m` 的基做正交化（工作量为 `O(n × m)`）。唯一的稠密特征分解规模是 `m × m`（`m = basis_dim`，默认 48，可选 16–200），因此 `n × n` 算子在整个流程中从不被构造或稠密化。

量	表达式	`n = 102400`、`basis_dim = 48`（实矩阵）时的值
算子存储	CSR，`nnz × (值 + 索引)`	6.5 MB（`nnz = 510720`，实值）
Krylov / 搜索空间基块	`n × basis_dim × 8`（复矩阵 16）	39.3 MB

量	表达式	`n = 102400`、`basis_dim = 48`（实矩阵）时的值
投影矩阵	`basis_dim²` 量级	约 18 KB
内层基块	`n × restart × 8`（`restart = 80`，仅位移逆）	65.5 MB

`solver.py` 按 `n × basis_dim × (16 或 8)` 计算并写入 `diagnostics.memory_hint_mb`；`csr.py: sparse_memory_bytes()` 给出稀疏存储的估算，各样例的该值在第九节的表中列出。内存的主要旋钮是 `basis_dim`（界面上标注为"基宽（内存档位）"）：它同时决定基块内存、每轮正交化开销与收敛速度。

不得稠密化是本插件的一条硬约束，代码层面表现为三点：CSR 构造路径（`csr_from_coo` / `_coo_reduce`）只使用 `O(nnz log nnz)` 的工作区；`NumpyCSR.dot()` 对多列右端项按列迭代（峰值内存 `O(nnz + n·k)`），不生成中间稠密矩阵；输入侧的稠密通道有显式护栏 `max_dense_cells = 4×10⁶`，超限直接抛 `MatrixIOError` 而不是尝试分配。

六 并行与运行环境

同一份求解器源码在两种环境下运行，通过 `backend.py` 在导入期探测：

环境	稀疏存储	后端标识	线程
CPython（本地 CLI / 脚本）	`scipy.sparse.csr_matrix`	`scipy <版本> (CPython)`	NumPy/BLAS 自动多线程（`OMP_NUM_THREADS` / `MKL_NUM_THREADS`）
浏览器（Pyodide）	SciPy 可用时 `scipy.sparse.csr_matrix`，否则 `NumpyCSR`（纯 NumPy）	`HAVE_SCIPY` 真则 `scipy <版本>…`，否则 `numpy-only (Pyodide degraded mode)`	WASM 单线程

浏览器侧现已与 numpy 一并 vendored 并加载 scipy（`em-worker.ts` 执行 `py.loadPackage(['numpy','scipy'])`，scipy wheel 位于 `public/pyodide/scipy-*.whl`，Pyodide 自建通道从本地加载而非 CDN）。因此网页运行时 `backend.py` 的 `HAVE_SCIPY` 通常为真，稀疏存储走 SciPy；只有 scipy 加载失败或缺失的退化情况下才回落到纯 NumPy 的 `NumpyCSR`——这不是故障路径而是设计上的兜底：`csr.py` 以手写 CSR 支持求解器所需的全部操作（`A @ x`、`A.H`、`diagonal`、`coo_of` 等），两个后端暴露同一套鸭子类型接口，数值结果一致。

关于并行加速，需要如实说明当前边界：**没有 MPI 路径**。多线程能力来自两处：NumPy/BLAS 的向量内核（由 `matvec` 与小型 `eigh` 调用自动获得），以及纯 NumPy 后端（`NumpyCSR.dot`）的行块多线程 matvec——大矩阵按行块切分后经 `ThreadPoolExecutor` 并行（NumPy 的大 ufunc 会释放 GIL），`EM_EIGENSOLVER_THREADS` 环境变量可覆盖线程数。这里要注明：**行块多线程只存在于纯 NumPy 降级后端**（对应浏览器缺 scipy 的兜底路径），SciPy 后端（本地 CPython 与浏览器已加载 scipy）则依赖 NumPy/SciPy 自带的向量线程化。实测拐点：早先把阈值临时下调、强制小规模走并行分支的诊断实测到 10 201 行仅 0.25×（线程池提交/同步开销反而拖慢）；按生产路由实测，线程池自 40 000 行起启用、收益温和（约 1.06–1.4×@4e4–1e5，见 9.4 节），因此默认行数阈值 `_PARALLEL_MIN_ROWS = 40000` 以下保持串行——线程池只在 $n \gtrsim 4\times10^4$ 启用。

MatVec 层另一条独立优化是块稀疏乘（`lobpcg.py::_mat_cols`）：对支持二维右端项的算符，一次性 `A @ X`（`X` 为 `n × B` 块）只遍历一次 CSR 索引，把 `B` 次单列调用合并为一次块乘，避免 Python 层逐列的重复取列、栈帧与内存流量。实测在 CPython + SciPy 后端、块宽 `B = 10` 下，这条路径相对逐列调用有 1.2–3.8× 的收敛前减速收益（n = 1e4 约 2.4×，n = 1e5 约 3.8×，见 9.4 节）——它不改变计算量，只压缩每轮迭代的 matvec 墙钟时间。纯 NumPy 降级后端（在绝不加载 scipy 的退化路径，如测试刻意屏蔽 scipy 导入时）的单列实现曾测得二维 `reduceat` 块内核反而比逐列慢 1.1–1.8×，故该后端保持逐列路径不回退块化，两条路径数值结果完全一致。

WebGPU 方向：宿主平台层（`src/core/wgsl.ts` + `src/core/gpu-kernels.ts`）提供了 CSR SpMV 计算内核（每行一线程、6-binding 布局、GPU 不可用自动回落 CPU），与 matmul/FFT/k-means/binning 同属平台级 GPU 资产并有完整 vitest 覆盖。对外口径一句话：**GPU SpMV 内核已实现并暴露为独立 API，求解主循环保持 CPU f64 精确路径**——Pyodide 中的 Python 求解是同步的，事件循环在求解期间被阻塞，而 WebGPU 的 readback 是异步的，逐 matvec 的同步 GPU 委派在当前架构下不可行，因此浏览器内运行的求解循环目前用不上它。Python 侧为此保留了通用同步桥钩子（`csr.set_gpu_spmv`），供未来的同步宿主接入；`SolverConfig.gpu_spmv` 打开时诊断信息会如实说明这一边界。相应地"缺少特定 GPU 硬件时的降级问题"不存在——GPU 不是运行前提，任何环境的行为一致。

七 插件运行时架构

7.1 文件职责

文件	职责
`index.ts`	工厂导出，供内置注册表惰性加载
`plugin.ts`	`Plugin` 接口实现：清单、参数定义与更新、求解编排、状态机、导出与 Figure Studio 桥接
`types.ts`	插件 ⇄ 客户端 ⇄ Worker 的协议类型（配置、请求、事件、结果载荷、模式场）
`em-client.ts`	Worker 生命周期与 RPC：首次使用时 spawn、串行队列、进度转发、终止与重建
`em-worker.ts`	Pyodide 引导、把 Python 源码写入解释器文件系统、进度回调桥、求解与导出入口
`render.ts`	2D 报告面板（谱、残差、收敛轨迹）；几何辅助为纯函数
`render3d.ts`	3D 模式场位移曲面与极值标注；色标、极点定位、缓冲构造为纯函数
`python/em_eigensolver/`	求解器包（14 个模块，见下；浏览器端加载除 `cli.py` 外的 13 个，见 7.2）

Python 包按单一职责拆分：`backend.py`（后端探测）、`csr.py`（纯 NumPy CSR 与稀疏原语，含行块多线程 matvec 与 GPU SpMV 同步桥钩子）、`io_matrix.py`（`.mtx` / `.npz` / `.npy` 读写与本地 `.h5` / `.fits` / `.nc` 可选直读、三角恢复、厄密化、结果写出）、`minres.py`（内层求解器）、`lanczos.py`、`lobpcg.py`、`jacdavid.py`（三个内核）、`samples.py`（参数化样例）、`solver.py`（门面与配置）、`sweep.py`（参数扫描诊断，见 9.5 节）、`driver.py`（JSON 桥）、`repro.py`（复现凭证，见 11.4 节）、`cli.py`（命令行）、`__init__.py`（包导出）。

7.2 数据流

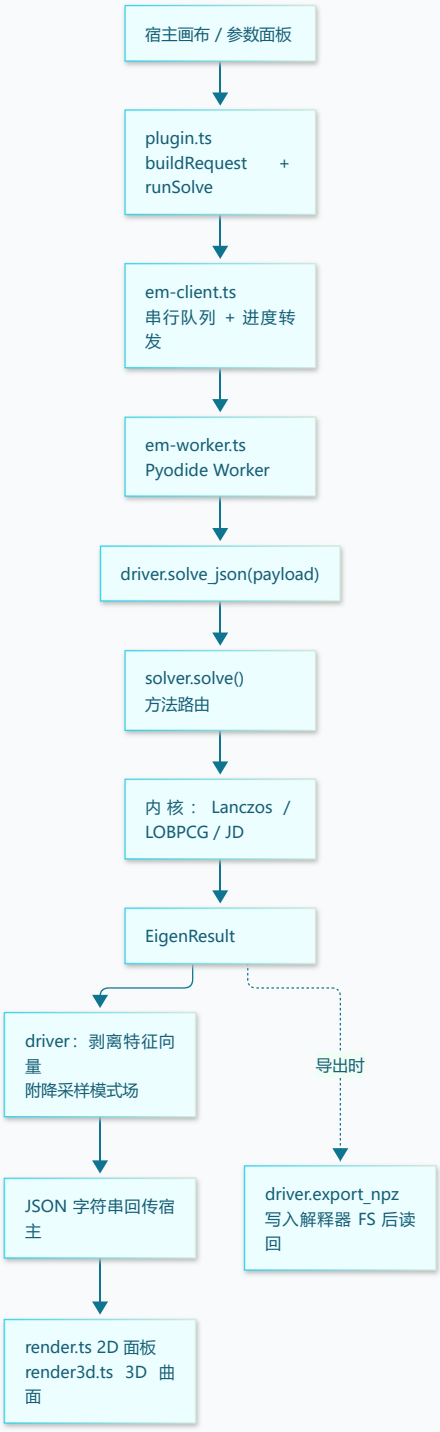

宿主画布 / 参数面板
plugin.ts
buildRequest + runSolve
em-client.ts
串行队列 + 进度转发
em-worker.ts
Pyodide Worker
driver.solve_json(payload)
solver.solve()
方法路由
内核：Lanczos / LOBPCG / JD
EigenResult
driver：剥离特征向量
附降采样模式场
导出时
JSON 字符串回传宿主
driver.export_npz
写入解释器 FS 后读回
render.ts 2D 面板
render3d.ts 3D 曲面

图 2

7.3 跨语言边界与生命周期

跨边界只传 JSON 字符串。浏览器 Worker 的 `postMessage` 无法结构化克隆 PyProxy 对象，因此 Python 侧的进度、报告、元数据一律序列化为 JSON 文本；`n × k` 特征向量块留在 Python 进程内，只在显式导出时写成压缩 `.npz` 再从解释器文件系统读回。这一约定把最大的数据块排除在消息通道之外——报告本身只带特征值、残差、诊断与降采样后的模式场。

几处与稳定性相关的设计：

- **引导与超时**：Worker 首次使用时创建，`em-worker.ts` 动态 import `pyodide.mjs`（模块 Worker 不允许 `importScripts`），加载 `numpy` 包，再把除 `cli.py` 外的 13 个 Python 源码文件写入 `/lib/em_eigensolver/`、把 `/lib` 插入 `sys.path` 并导入。宿主侧引导超时为 120 秒；引导失败可重试（失败时清空缓存的 bootstrap Promise）。
- **进度桥**：Python 端 `driver.set_progress_sink()` 接受一个 JS 回调，每个重启周期把进度字典 JSON 化后推回宿主；进度回调中的异常被吞掉，保证"进度上报永不影响求解"。
- **串行队列**：客户端把求解与导出请求排在一条 Promise 链上，同一时刻只跑一个作业；作业内再次校验 Worker 身份，避免已被终止的实例继续返回结果。
- **终止语义**：用户点"终止求解"时不做软取消，而是**整体重建计算栈**——终止 Worker、重建客户端、复位状态栏与按钮文案，下一次求解会重新启动解释器。求解期的陈旧拒绝（abort 之后到达的异常）用自增的 epoch 号识别并直接丢弃，不会污染当前状态。
- **渲染与场景**：3D 视图需要的宿主场景句柄在 `renderToScene` 时获得；宿主场景在工作台卸载（例如切到 Figure Studio）后会被重建，因此缓存的网格除了按模式场判断，还要按场景句柄判断是否需要重建。切回 2D 报告时主动隐藏 3D 面并释放网格与标注资源。

八 界面与交互

8.1 参数面板

参数面板由清单驱动，宿主按当前语言自动生成本地化控件。数值型参数一律做成**下拉选择**而非自由数字输入，既限定了有意义的档位，也避开了"默认值无法删除"的输入框陷阱。

控件	键	取值
数据来源	`matrix`	内置样例 6 项（含内置示例文件）+ 导入文件项（仅在已导入时出现）
算法	`method`	自动 / 厚重启 Lanczos / 块 LOBPCG / Jacobi-Davidson
特征值个数 k	`k`	1、2、3、4、5、6、8、10、12、16、20、24、32、48、64
位移 σ	`sigma`	文本，留空表示求极端特征值
收敛容差	`tol`	1e-4、1e-6、1e-8、1e-10、1e-12
基宽（内存档位）	`basisDim`	16、24、32、48、64、80、96、128、160、200
随机种子	`seed`	0–9
视图	`view`	求解报告（2D 面板）/ 3D 模式场
谐振模式（3D 视图）	`modeIndex`	每个已返回模式一项，标签带 λ 值
运行求解 / 终止求解	`run` / `abort`	动作按钮
导出 .npz	`exportNpz`	动作按钮（有结果后出现）
导出诊断报告	`exportReport`	动作按钮（有结果后出现）
导出复现凭证	`exportRepro`	动作按钮（有结果后出现）
发送到 Figure Studio	`sendToFigure`	动作按钮（有结果后出现）
重置插件	`reloadPlugin`	动作按钮

数据来源采用单层下拉（`sample:<id>` 或 `file`），取代了早期"矩阵来源 + 样例"两级选择；旧的 `source` / `sample` 键仍然接受，以便已保存的项目与顶栏样例导入继续可用。这里有一条务实的约束：`file` 选项只有在 `File` 对象确实存在时才生效——`File` 不随页面刷新保留，恢复一个只有 `source: 'file'` 而没有文件的项目时必须回落到样例路径，否则下拉框会显示一个点了必然失败的选项。

请求配置由 `buildRequest()` 组装：`which` 固定 `LM`，`maxCycles = 60`、`maxIter = 400`、`denseThreshold = 800`，其余取自界面状态。键名同时接受 camelCase（界面）与 snake_case（CLI）两种命名（如 `maxCycles` / `max_cycles`），由专门单测（`test_driver_config_accepts_camel_case`）锁定这一兼容约定。

8.2 2D 求解报告

求解前显示进度行与内核日志尾部（最多 8 行），以及一张实时收敛迷你图；得到结果后画布分为三个面板：

面板	内容
谱（上部，占高度 55%）	升序排列的特征值散点，横轴画 5 个刻度，`σ` 以虚线标出并带标签
相对残差（下部左，占宽度 45%）	每个特征值一根柱，按是否达标取不同颜色，叠加对数刻度的容差虚线
外迭代收敛轨迹（下部右，占宽度 55%）	每轮的最大相对残差随周期数变化的折线，纵轴对数，横轴约 8 个刻度

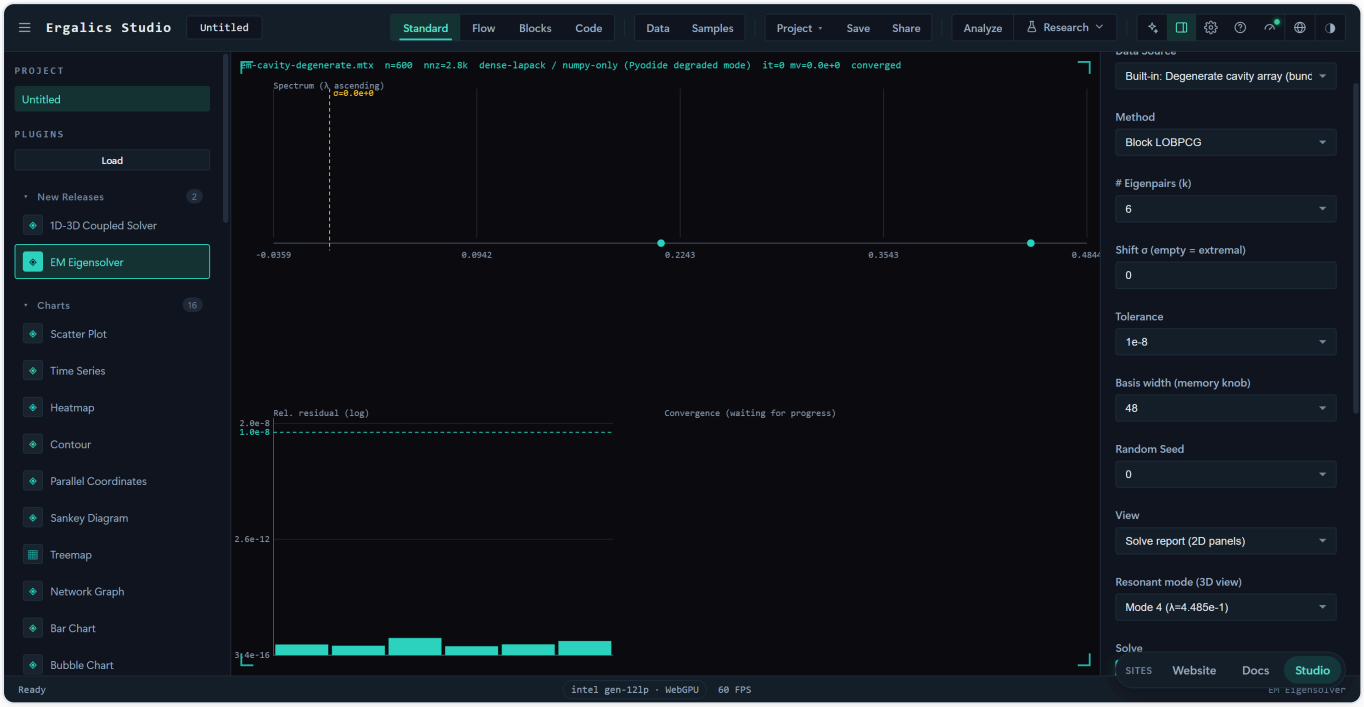
顶部信息行为"样例名 / n / nnz / 算法 / 后端 / 迭代数 / matvec 数 / 是否收敛"。所有几何辅助（谱域计算、值到像素映射、刻度选取、进度摘要）都是导出的纯函数，便于单元测试覆盖。

参数面板在求解期间同时承担进度显示：画布切到实时收敛轨迹（当前迭代步、最大相对残差、matvec 计数），面板底部按钮变为"计算中…"，"终止求解"保持可用。



求解进行中的画布：实时显示当前收敛轨迹（iter / rel / mv），参数面板按钮变为"计算中…"并以终止按钮保持随时可中断，便于观察迭代进展与响应式交互。

求解完成后的报告如下：上部是升序谱散点并以虚线标注 σ，左下是每个特征值的相对残差柱与对数刻度容差线，右下留给外迭代收敛轨迹；顶部信息行给出样例名、n、nnz、算法、后端、迭代数与是否收敛。



求解完成报告（稠密直解路径）：上部升序谱散点并以虚线标注 σ，左下相对残差柱叠加对数容差线；该规模走稠密直解、没有外迭代，故右下收敛轨迹面板保持为空并显示等待提示。

当样列规模不具备稠密直解条件而走块 LOBPCG 等迭代路径时，右下的外迭代收敛轨迹会被真实填充，形成下述的收敛报告：



LOBPCG 路径的收敛报告：右下外迭代面板真实显示每轮最大相对残差随周期数的下降轨迹，顶部信息行给出迭代数（如 161 次）与 matvec 计数，供核验收敛行为。

8.3 3D 模式场

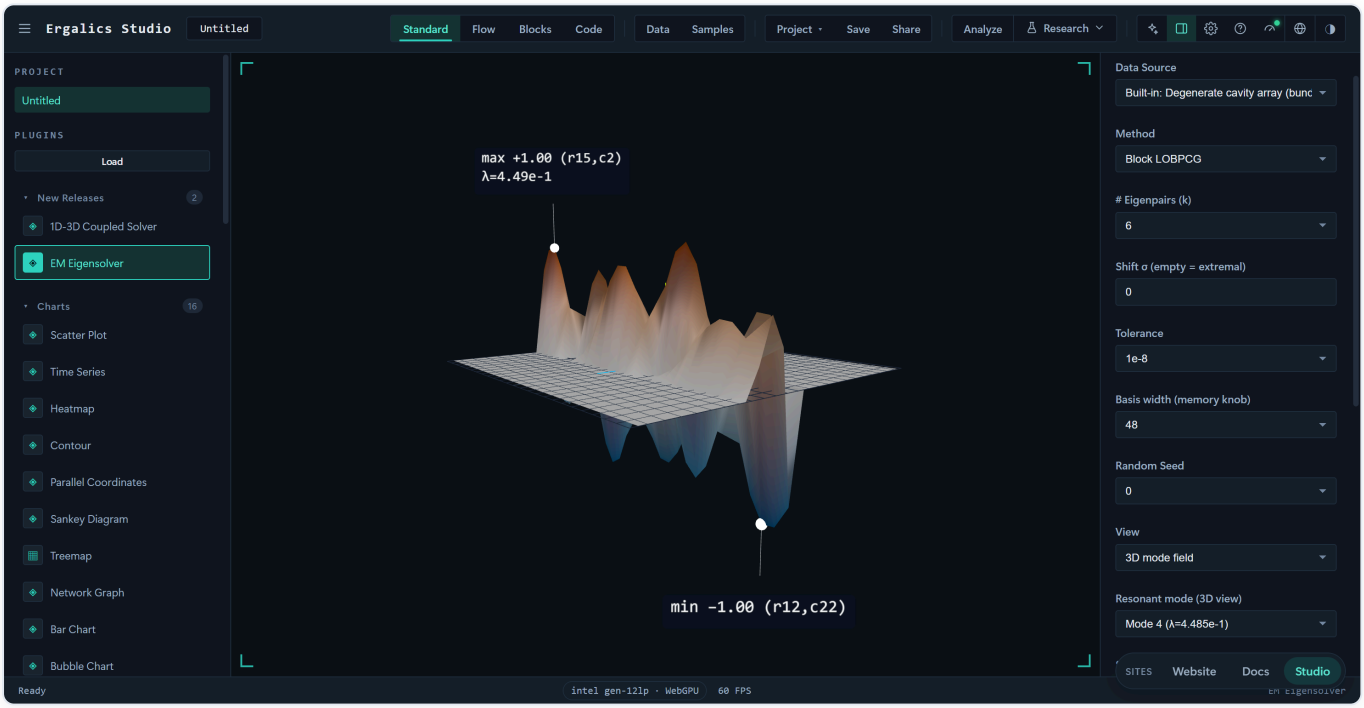
3D 视图把选中的模式场画成位移曲面：网格值同时决定高度与顶点颜色，色标采用 Okabe-Ito 发散配色（负值走蓝、零为白、正值走朱红），并在正峰与负谷各放一枚标记球、一条垂线与一个浮层标签（含数值、网格坐标 `(r, c)`，主导侧额外带 λ）。顶点颜色在写入前做了 sRGB → 线性空间转换，否则渲染器输出时再次应用线性到 sRGB 的变换会把中间色调冲成粉彩、与 SVG 导出不一致。

两个与宿主相关的细节：`heightScale` 固定 0.35，高度按 `值 × heightScale × max(rows, cols)` 缩放，使不同网格尺寸下的观感一致；相机距离按网格外接尺寸计算（`extent × 2.1 + 2`），并在每次重建网格时重新取景。

模式场数据本身由 `driver.mode_fields()` 生成：对最后一次求解的前 6 个特征向量做**块均值降采样**，每轴最多 64 格（等距分块，最后一块可能更宽），复特征向量取模、实特征向量保留符号（以便看清节线结构），整体归一化到最大绝对值 1。网格形状按 `n` 的最近因子对推断：`cavity_*` 这类方形网格样例能还原真实网格（`approx: false`），而按波段拼装的样例只能得到近似布局（`approx: true`），报告中的标志位如实反馈这一点。质数（以及 n < 4 的极小值）没有任何两侧都 ≥ 2 的因子对——若强行取 `(1, n)` 布局，曲面网格将退化成零 quad 的空片；因此 `_nearest_grid` 对这类 n 回退为近似 2 行布局，`mode_fields` 对多余格点零填充，仍以 `approx: true` 标记。

需要提醒的是，Pyodide 运行在 Worker 中，而 3D 渲染需要主线程的宿主场景；若场景未挂载，插件会退回 2D 报告并给出一次提示（按原因去重），引导使用者"重置插件"后重新求解。

下图是内置示例文件 `em-cavity-degenerate.mtx` 求解后第 4 个模式（λ = 4.485e-1）的模式场：曲面高度与顶点颜色来自同一份降采样网格，正峰与负谷各有一枚标记球、一条垂线与一个浮层标签，标签中含峰值、网格坐标 `(r, c)` 与 λ。



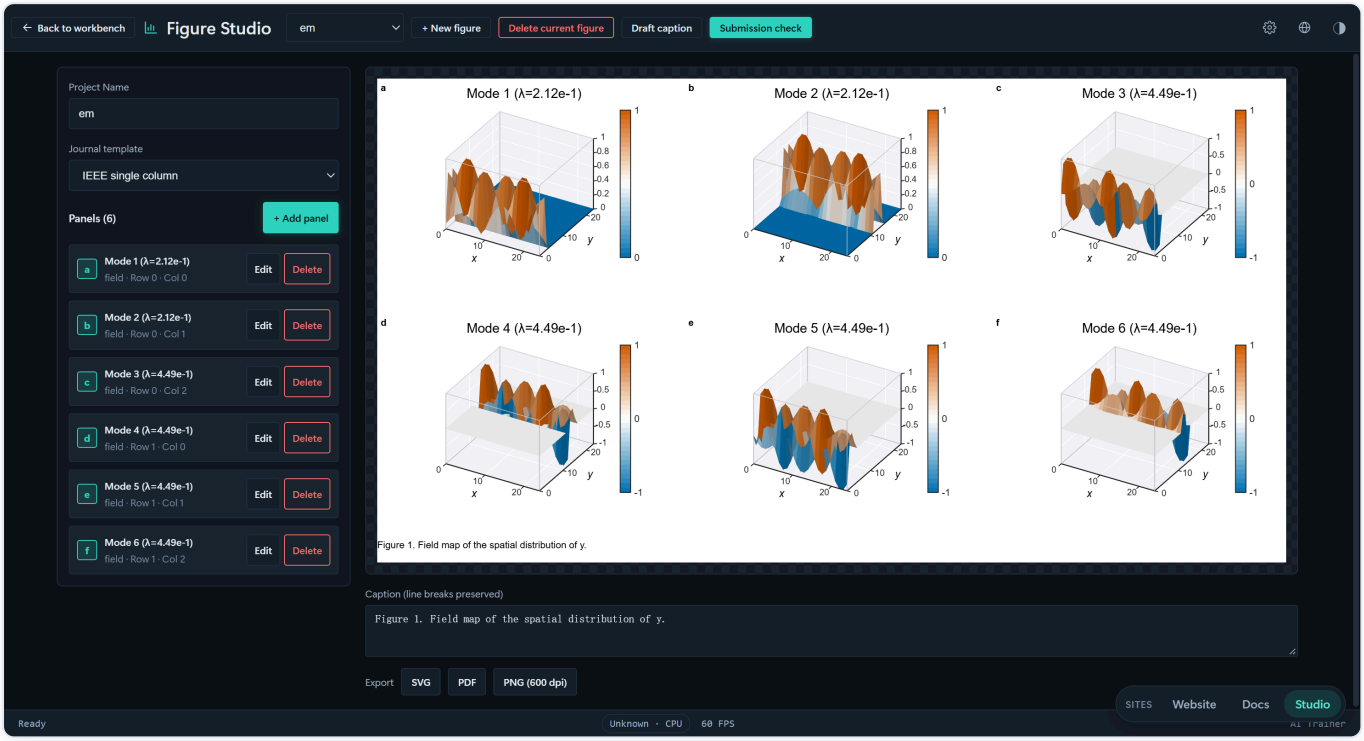
3D 模式场（内置简并示例）：第 4 个模式曲面的高度与颜色来自同一份降采样网格，峰 +1.00、谷 −1.00，正峰负谷各带标记球、垂线与浮层标签（含峰值、网格坐标与 λ）。

小规模样例 `cavity_small`（30 × 30）配合块 LOBPCG 内核时，同一视图给出极端特征值附近的模式场，即文档开头的 3D 模式场曲面。

8.4 与工具链的衔接

- **导出 `.npz`**：把最后一次求解的特征值与特征向量写成 NumPy 归档（`eigenvalues`、`eigenvectors`、`residuals`（真实残差，与 CLI 输出一致），可选 `meta` 为 JSON 文本框；虚部整体小于 `1e-14` 的复特征向量会自动降为实数组），文件名沿用输入名加 `.eigen.npz`。
- **导出诊断报告**：一键生成自包含 HTML（`ergalics.em-diag-report`，`<名字>.diag-report.html`），同页内联 SVG 呈现求解参数、矩阵信息、特征值谱（含 σ 标记）、逐特征对相对残差条形图（对数轴 + 容差线）、外层收敛轨迹与全部模式场热图（发散色标：蓝负红正）。零脚本、零外部资源，收件人离线双击即可打开，是"工程诊断输出"的可交付形态。
- **导出复现凭证**：把随每次求解报告生成的可复现凭证（`ergalics.em-repro`）原文下载为 `repro.json`——矩阵指纹 + 参数哈希 + 种子 + 代码快照 + 结果摘要（详见 11.4 节）。
- **发送到 Figure Studio**：每个模式场生成一个 3 列排布的图表面板，标签按电子表格习惯编号（a、b、c…），带发散色标与 λ 标题，并写入整张图的图注。图仓库在此处惰性导入——静态导入会形成 `plugin → figureStore → projectStore → pluginStore → builtin registry → plugin` 的循环，使注册表条目在初始化时未定义。
- **顶栏内置示例**：`examples/data/em-cavity-degenerate.mtx` 同时登记在全局示例库（id `em-cavity-degenerate`，绑定到本插件），因此从顶栏导入与从插件下拉选择走的是同一条文件路径。

下图是"发送到 Figure Studio"的结果：6 个模式场以 2 × 3 面板排布，套用 IEEE 单栏模板，每个面板带发散色标与含 λ 的标题，绘图区下方由插件写入一行图注，供使用者续写。这批模式取自内置简并示例，标题里的 λ 恰好重复出现（2.12e-1 两次、4.49e-1 四次），与 9.2 节实测的重数分布互相印证。



发送到 Figure Studio 的组图：6 个模式场以 2 × 3 面板排布，套用 IEEE 单栏模板并写入整张图注；每个面板带发散色标与含 λ 的标题，标题里的 λ 重数分布与实测简并情况互相印证。

九 内置样例与基准数据

9.1 五个参数化样例

样例由 `samples.py` 的构造器生成，全部是稀疏、按构造即厄密的矩阵，尺寸与参数由 `(n_cells, bandwidth, seed, ...)` 之类的旋钮决定，没有把任何题面尺寸或答案写死。下表数据为按默认参数实际构造后测得。

样例	n	nnz	CSR 存储	推荐 σ	推荐 k	谱范围（实测）	覆盖场景
`cavity_small`	900	4380	0.06 MB	—（极值）	6	0.5153 … 8.4847	小规模冒烟测试、低内存环境
`cluster_zero`	720	2148	0.03 MB	0.0	8	−0.2836 … 0.3040	零附近密集谱 + 近奇异位移
`degenerate_pair`	300	894	0.01 MB	−1.0	6	−1.2511 … 2.2942	重特征值（收缩与块迭代）
`cavity_complex`	576	2784	0.06 MB	1.2	6	1.0282 … 8.9718（复厄密）	复厄密算子
`cavity_large`	102400	510720	6.54 MB	—（极值）	6	五点差分网格，理论范围 0 … 8	十万阶、控内存

各样例的构造方式：

- `cavity_small`、`cavity_complex`、`cavity_large` 是二维五点差分网格 Laplacian（`_grid_laplacian`），在被移位的谱上加入确定性的材料不均匀项；`cavity_complex` 进一步给水平耦合加上一个常数相位 `e^{iϕ}`，矩阵仍保持厄密。
- `cluster_zero` 由 6 个窄带三对角块拼装，块中心在 ±0.2 之间等距分布、半宽 0.03，共 720 个特征值挤在零附近。实测其中 153 个特征值满足 `|λ| < 0.05`、35 个满足 `|λ| < 0.01`，因此默认的 `σ = 0` 确实落在密集谱内部，`A − σI` 接近奇异。
- `degenerate_pair` 由三个三对角块拼装，其中前两个块是同一个块（同一数组重复两次），因此该带的每个特征值恰好二重。

`cavity_large` 是"十万阶"量级的实证用例：`n = 102400`（320 × 320 网格）、`nnz = 510720`，CSR 存储约 6.5 MB；按默认基宽 48 计算，Krylov 基块约 39.3 MB，与仓库文档给出的数百兆量级预算相符。

9.2 内置示例文件（实测结构）

`examples/data/em-cavity-degenerate.mtx` 是随仓库提供的一个 Matrix Market 文件，登记在全局示例库并由插件下拉直接选用。对其做了独立解析与谱分析，实测结构如下：

属性	实测值
维度与规模	600 × 600，`nnz = 2760`（实数，对称）
结构	块对角：6 个 100 × 100 块，每块 `nnz = 460`；块间无耦合
块内容	每块为 10 × 10 网格的标准五点差分 Laplacian：对角常数分别为 4.05、4.6、5.4，非对角元均为 −1
重复方式	三种参数各出现两次（块 0≡3、1≡4、2≡5），因此每个特征值至少二重
单块谱	100 个特征值中 51 个不同；重数分布为 1（10 个）、2（40 个）、10（1 个），来自网格的 p↔q 对称
全局谱	600 个特征值中 153 个不同，范围 0.2120 … 9.2380；重数分布为 2（30 个）、4（120 个）、20（3 个）
默认求解路径	600 ≤ `dense_threshold`(800)，因此走稠密直解，报告显示 `dense-lapack`

文档与数据曾有一处不一致：全局示例库对它的描述是"3 组不同参数的 10×10 谐振腔各重复两次，每个特征值恰好两重"。前半句与实测一致，后半句不准确——由于 10 × 10 网格本身的谱就含重数 2 与 10 的内部简并，整体谱的重数是 2 / 4 / 20 三种（都是偶数重，但并非"恰好两重"）。示例描述已按实测重数分布更正为"实测特征值重数分布为 2 / 4 / 20（均为偶数重）"（中英双语同步）。

9.3 基准与复现

三个环境使用同一份样例构造代码，因此基准可比：

```
# 本地 CPython（SciPy 后端）
cd src/plugins/builtin/em-eigensolver/python
python -m em_eigensolver.cli --sample cluster_zero --sigma 0 --k 8 --verbose --out eigen.npz
python -m em_eigensolver.cli --sample cavity_large --k 6 --config config.example.json --out large.npz

# 浏览器：插件面板选择样例与参数，点"运行求解"
```

CLI 在标准输出给出后端、输入描述、方法、位移、k、收敛状态、迭代数、matvec 数、最大相对残差、耗时与特征值列表，并把特征值、特征向量与**真实残差**写入 `--out` 指定的 `.npz`（`eigenvalues` / `eigenvectors` / `residuals` 键；`meta` 键内附 JSON 元数据，含方法、后端、迭代数、matvec 数、耗时、位移与内核诊断）。`--config` 可加载完整配置 JSON，`config.example.json`（位于 `python/` 下）给出全部字段与默认值。`basis_dim` 等未暴露为独立命令行参数的字段需经 `--config` 设置，CLI 无 `--basis_dim` 之类的直传开关。退出码约定：收敛为 0，未收敛为 1，配置或输入错误为 2。

参数扫描（`--sweep KIND:PARAM=V1,V2,...`）把单点求解升级为诊断曲线：例如 `--sweep cavity:size=12,16,20,24` 逐点重建矩阵、求解并输出 JSON 报告（每点的特征值、真实残差、收敛状态、耗时与误差捕获），细节见 9.5 节。

`--repro PATH` 在求解后额外写一份可复现凭证（`repro.json`，`ergalics.em-repro` v1）：矩阵指纹 + 参数哈希 + 种子 + 代码快照 + 结果摘要。第三方用同一输入重跑后比对凭证中的 `matrix.hash` / `params_hash` / `result.eigenvalues_hash` 即可断言复现（详见 11.4 节）。浏览器端插件同样可以导出：每次求解报告自带凭证，点"导出复现凭证"下载。

需要提醒的是，`--sample` 路径在没有显式 `--sigma` / `--config` 时会套用该样例的推荐位移：推荐 σ 为 `-`（极值）的样例（`cavity_small` / `cavity_large`）默认走极端特征值路径（`which=LM`、Lanczos，秒级收敛），有推荐位移的样例（`cluster_zero` / `degenerate_pair` / `cavity_complex`）则按各自推荐 σ 进入 `auto` 路由（谱内目标交给 Jacobi-Davidson）。这与插件面板的默认行为一致。

9.4 性能实测（3.2 / 3.3）

以下数据由 `python/benchmarks/perf_bench.py` 在本机实测产出（Windows、8 逻辑核、CPython 3.x + numpy 2.2.6 + scipy 1.15.3；SpMV 曲线与两条求解曲线都走 `perf_bench.py` 的纯 NumPy CSR 强制路径 `_numpy_csr_grid`，因此求解链路自带的行块多线程 matvec 也被计入；`best-of-5` 计时，求解含 tracemalloc 峰值内存），原始 JSON 落在仓库 `bench/em-eigensolver-results.json`，块 MatVec 对比（SciPy 后端）的原始 JSON 落在仓库 `bench/em-eigensolver-block-matvec.json`。

SpMV 行块多线程曲线（纯 NumPy 后端，按生产线程路由实测：行块线程池仅在 `n ≥ _PARALLEL_MIN_ROWS = 40000` 时启用，不做任何强制）：

网格 n	nnz	实际路径	耗时	加速比
1 024	4 992	串行	0.026 ms	1.00×
4 096	20 224	串行	0.063 ms	1.00×
10 201	50 601	串行	0.385 ms	1.00×
40 000	199 200	并行（8 线程）	1.127 ms（串行 1.19 ms）	1.06×
102 400	510 720	并行（8 线程）	2.682 ms（串行 3.791 ms）	1.41×

诚实结论：`reduceat` 行块向量化本身已经很快，小规模下线程池的提交/同步开销远超收益——把阈值临时下调、强制 10 201 行走并行分支的诊断实测只有 0.25×（比串行慢 4 倍），这正是默认阈值 `_PARALLEL_MIN_ROWS = 40000` 的由来：线程池只在 $n \gtrsim 4\times10^4$ 启用，其上有约 1.06–1.41× 的温和收益（实测受机器负载波动，1e5 时 1.09–1.41×），以下保持串行（生产求解在 4 万行以下本来就不会触碰线程池，因此上表前三位就是真实求解会得到的性能）。

块 MatVec 对比（CPython + SciPy 后端、块宽 B = 10，`lobpcg.py::_mat_cols` 的块路径相对逐列单列调用；两端数值逐位一致）：

网格 n	nnz	一次 `A @ X`（B=10）	B 次单列调用	加速比
grid-100×100	49 600	0.175 ms	0.422 ms	2.42×
grid-200×200	199 200	1.31–1.41 ms	1.61–2.02 ms	1.23–1.44×
grid-320×320	510 720	3.080 ms	11.701 ms	3.80×

求解规模曲线（厚重启 Lanczos，k=6，tol=1e-8，extremal，纯 NumPy CSR 路径，含行块多线程 matvec）：

样例	n	收敛	最大真实残差	周期	matvec	耗时	峰值内存
grid-32×32	1 024	是	7.2e-09	6	295	0.16 s	0.7 MB
grid-64×64	4 096	是	1.3e-08	13	631	0.49 s	2.4 MB

样例	n	收敛	最大真实残差	周期	matvec	耗时	峰值内存
grid-101×101	10 201	是	1.4e-08	13	631	3.05 s	5.8 MB
grid-200×200	40 000	是	8.4e-09	25	1 207	12.8 s	22.5 MB
grid-320×320	102 400	是	2.1e-08	25	1 207	32.5 s	57.4 MB

cavity_large（n = 102400）两条路径对照（headline 为 extremal 行；容差口径随行归档为 `tol_requested` / `tol_effective` / `certified_rel_residual` 三字段，残差指绝对值 $||Ay - \lambda y||_2$）：

路径	收敛	最大真实残差（相对认证值）	matvec	耗时	峰值内存
extremal（`which=LM`，Lanczos）	是	3.4e-08（4.3e-09）	1 207	25.2 s	57.4 MB
内点位移逆（`σ = 0.5`，对照行）	是	5.9e-05（7.4e-06）	41 017	797.2 s	122.2 MB

同一矩阵、同一 k 下的对照需要按两个口径正确解读：

1. **容差口径。**位移逆行请求 `tol = 1e-8`，但内层 MINRES 有意不精确（`rtol = 1e-6`），外层有效容差放宽为 `max(tol, 20 × minres_rtol) = 2×10⁻⁵`。残差是绝对值 $||Ay - \lambda y||_2$，收敛判定按相对值除以返回的最大 `|λ|`：5.9e-05 / 8.04 ≈ 7.4e-06 ≤ 2×10^{-5}，故 `converged = true` 自洽——但请求容差与认证容差是两回事，基准 JSON 以 `tol_requested` / `tol_effective` / `certified_rel_residual` 分字段归档，绝不混同（见 2.3 节）。
2. **自适应位移逃逸。**这条对照行收回的**并非 σ = 0.5 邻域的特征对**——返回的特征值是 8.0424 …（谱顶）。密集谱内点使内层 MINRES 天然高负载，触发自适应位移把 σ 按步长倍增逐步移出谱：同一现象在 n = 4096 的诊断运行中被完整记录（`shift_history`：0.5 → 0.815 → 1.446 → 2.708 → 5.231 → 10.277，越出谱顶 8.51），外层最终收敛到距终态 σ 最近的**谱端**认证特征对。这正是 2.1 节方法路由把"有 σ 的谱内目标"默认交给 Jacobi-Davidson 的原因（JD 的投影校正方程天然规避近奇异，不依赖位移逃离），也是第十四节把"密集谱内点的位移逆"列为机制性边界、建议改用 JD 的依据。

因此 headline 采用 extremal 行：25.2 秒、1 207 次 matvec 认证 4.3e-09（相对），峰值 57.4 MB——"不稠密化 + 内存随 nnz 线性 + 十万阶可收敛"的证据链完整；位移逆行仅作对照，用于说明谱内内点目标的正确打开方式是 JD。

9.5 参数扫描诊断（4.2）

`sweep.py`（CLI 经 `--sweep KIND:PARAM=V1,V2,...` 触发，Worker 经 `sweep_json` 桥）把单点求解升级为工程诊断曲线：扫一个几何/材料旋钮，逐点重建矩阵、求解、记录，输出"参数 → 谐振频率带"的轨迹视图。

旋钮	适用样例	含义	默认扫描值
`size`	cavity 族	方网格边长（`n = size²`）	12, 16, 20, 24
`mu`	cavity 族	材料偏置，整谱平移	0, 0.5, 1, 1.5
`scale`	cavity 族	耦合强度	0, 0.5, 1, 1.5
`seed`	其余全部样例	重建种子	0, 0.5, 1, 1.5

每点输出 `value`、特征值列表、真实残差、收敛状态、迭代数、matvec 数、独立计时、`n`/`nnz`；单点异常被捕获为该点的 `error` 字段而**不会杀死整条扫描**（配置错误仍会显式失败）。扫描完全可复现：同一参数列表 + 同一种子必然重建同一批矩阵。正确性由 `test_parameter_sweep_curves` 守住：size 曲线逐点收敛且残差 ≤ 1e-6、谱落在理论区间内；mu 旋钮以 `which='SA', k=1` 验证最小特征值的平移量恰好等于扫参步长（1.0）；非法旋钮报错。

9.6 正确性验证与对抗基准（4.3 / 4.4）

`python/benchmarks/validate_correctness.py` 一次运行产出两张表（`--fast` 跳过两个大用例；`--json` 落盘原始报告，仓库归档于 `bench/em-eigensolver-validate.json`）：

4.4 稠密参考对照——每个中小规模样例与 `numpy.linalg.eigh`（LAPACK）比对：按目标选取规则取同一批参考特征值，报告最大特征值误差与由参考向量过稀疏算子重算的残差。实测（scipy 后端）：

用例	方法	max|dλ|	最大残差	matvec	状态
cavity_small k=6（extremal）	lanczos	2.3e-14	2.5e-09	295	PASS
cavity_complex k=4（extremal）	lanczos	5.2e-14	8.3e-10	245	PASS
degenerate_pair k=4 σ=−1	jacobi-davidson	2.1e-13	9.9e-09	27 688	PASS

用例	方法	max|dλ|	最大残差	matvec	状态
cluster_zero k=4 σ=0	jacobi-davidson	4.1e-18	3.5e-09	40 137	PASS
cavity_small k=4 σ=8.3（谱顶内点）	jacobi-davidson	5.3e-15	4.4e-09	89 215	PASS

特征值误差全部在 1e-12 量级以下（多数落在 1e-14，对 LAPACK 的浮点一致），真实残差全部 ≤ 1e-8。

4.3 对抗 stress bench——八组数值对抗用例，全部以真实残差认证（或要求干净地失败）：

用例	考察点	最大残差	状态
near_singular_shift（σ=0，零附近密集谱）	近奇异位移 + 自适应	3.5e-09	PASS
dense_spectrum_interior（σ=0.05）	谱内密集目标	5.0e-09	PASS
repeated_eigenvalues（σ=−1）	重特征值收缩	9.9e-09	PASS
ill_conditioned_scale_1e9（矩阵 ×1e9）	极端尺度不变性	1.5e-09	PASS
ill_conditioned_scale_1e-9（矩阵 ×1e-9）	极端尺度不变性	7.3e-09	PASS
complex_hermitian（σ=1.2）	复厄密算子	2.8e-09	PASS
large_scale_cavity_1e5（extremal，n=102400）	十万阶收敛认证	8.9e-09	PASS
nonfinite_input（植入 NaN/Inf）	必须显式失败而非静默"收敛"	—（未收敛，按预期 flag）	PASS

`--fast` 子集（前四组参考对照 + 除 1e5 外的 stress 用例）由测试套件的 `test_validate_correctness_fast_subset` 随 CI 执行；完整版在本机实测 13 用例 0 失败、约 92 秒。参考对照里 `sigma=None` 严格表示 extremal 请求（不代推荐位移），返回特征值个数不足 k 时判 FAIL——验证基准不吞短返回。

故障场景与处置建议

症状	成因	处置
内层 MINRES 达到 `maxiter`	`σ` 几乎正好是一个特征值，`A − σI` 近奇异	自适应位移会自动介入；否则加大基宽或提高内层迭代上限
达到 `max_cycles` 仍未收敛	谱内密集簇而基宽不足	提高基宽、放宽容差，或改用 Jacobi-Davidson
位移逆 Lanczos 报告收敛但特征值远离 `σ`	谱内目标附近谱太密，内层持续高负载触发自适应位移把 `σ` 逐步移出谱（`diagnostics.shift_history` 呈倍增轨迹），最终锁定距终态位移最近的谱端特征对而非 `σ` 邻域目标	属机制性行为：谱内密集目标请改用 Jacobi-Davidson（`auto` 路由默认如此）；显式位移逆仅建议用于谱间隙内的目标
随机稀疏矩阵上进展缓慢	随机稀疏矩阵的谱近似 Wigner 半圆，任何谱内目标附近都很密	这是问题本身的性质而非缺陷；应改用有结构的样例或从物理问题而来的矩阵
选了 LOBPCG 但位移没生效	该内核只求极端特征值	需要谱内目标时改用 Jacobi-Davidson 或位移逆 Lanczos
选 `1e-12` 容差却停在约 `2e-5`	位移逆 Lanczos 的有效容差下限 `20 × minres_rtol`	放宽预期，或改用其他内核
未收敛且残差不下降	期望精度低于内层不精确求解所能提供的水平	提高精度需求时应改写问题而非常规调参
`MatrixIOError: unsupported extension`	核心格式只接受 `.mtx` / `.npz` / `.npy`；`.h5` / `.fits` / `.nc` 仅在本地 CPython 且装了对应可选包时可用	浏览器侧转换格式或改选内置样例；本地侧 `pip install h5py` / `astropy`（`.nc` 随 SciPy）
报告"结果包含非有限值"	求解发散产生 NaN/Inf，`driver` 清洗为 null 并置 `nonfinite` 标志，宿主拒绝渲染	调整位移 σ 或容差后重试
稠密文件被拒绝	超过 `max_dense_cells = 4×10⁶`	改用稀疏 `.npz` 或 `.mtx`
3D 视图退回 2D	宿主场景未挂载，或本次结果不含模式场	按提示重置插件后重新求解

十一 测试与可复现性

11.1 Python 测试套件

`python/tests/test_all.py` 共 26 项测试，可独立运行（`python tests/test_all.py`）或交给 pytest。覆盖范围：

测试	覆盖点
`test_csr_matvec_and_hermiticity`	纯 NumPy CSR 的 matvec 与厄密性度量、存储估算
`test_numpy_csr_backend_matches_scipy`	两后端一致性：经 `csr_from_coo` 构造 `NumpyCSR`，与 SciPy CSR 比对 matvec / 共轭转置 / 复合矩阵，误差 < 1e-12
`test_numpy_csr_parallel_matvec_matches_serial`	行块多线程 matvec 与串行结果逐位一致（含空行 CSR），线程数与阈值经 monkeypatch 强制走并行分支
`test_hermiticity_measure_defaults`	`hermiticity_measure` 无 `rng` 参数可调用、`sample_size` 生效为探针列数、非厄密矩阵测度显著大于零
`test_mtx_npz_roundtrip`	`.mtx` 与稀疏 `.npz` 的写读往返，误差 < 1e-12
`test_hermitize_and_materialize`	三角恢复与厄密化投影、漂移估计
`test_minres_indefinite`	不定系统的 MINRES 收敛与真实残差
`test_lanczos_plain_and_shift_invert`	极端特征值与位移逆两条路径，对照稠密 LAPACK 参考
`test_lobpcg_extremal_and_repeated`	极端特征值、重特征值块收敛、特征向量正交性
`test_jacobi_davidson_dense_interior_and_deflation`	谱内目标、简并模式收缩、正交性
`test_jd_inner_tolerance_is_relaxed`	JD 内层容差按"有意不精确"取 `max(minres_rtol, 1e-4)`（spy 校验）

测试	覆盖点
`test_samples_hermitian`	各样例构造的厄密性
`test_solver_facade_all_methods`	门面路由（含稠密兜底必须返回 `dense-lapack`）
`test_driver_config_accepts_camel_case`	驱动层双命名兼容：界面 camelCase（`basisDim` / `maxCycles` 等）与内部 snake_case 等价
`test_export_npz_contains_residuals`	导出归档含 `eigenvalues` / `eigenvectors` / `residuals` 三键
`test_parameter_sweep_curves`	参数扫描：size 曲线逐点收敛且残差达标、mu 平移恰好等于扫参步长、非法参数报错不杀死扫描（见 9.5 节）
`test_validate_correctness_fast_subset`	4.3 + 4.4 快速子集：稠密参考对照与对抗基准全部 PASS，特征值误差 < 1e-8
`test_cli_end_to_end`	CLI 端到端：写矩阵 → 求解 → 校验输出 `.npz`
`test_driver_solve_json_flat_payload`	Worker 桥的扁平载荷约定与报告结构（含嵌套载荷必须显式失败）
`test_driver_mode_fields`	模式场降采样：方形网格还原、带状样例近似布局、每轴 64 格上限、复特征向量取模
`test_nearest_grid_prime_falls_back`	质数/极小 n 的退化布局回退（两侧 ≥ 2 + 零填充），精确因子仍还原方形网格
`test_sanitize_json_replaces_nonfinite`	NaN/Inf → null 清洗与 `nonfinite` 标志，杜绝非法 JSON 进入宿主解析器
`test_repro_credential_determinism`	凭证确定性：同输入两次构建的全部摘要一致、所有内核模块参与代码快照、JSON 往返无损（见 11.4 节）
`test_repro_fingerprint_sensitivity`	矩阵指纹敏感性：不同矩阵/参数改变摘要、相同重建不变、稠密矩阵走稠密路径

测试	覆盖点
`test_driver_report_carries_repro`	Worker 桥报告自带凭证且可经 `export_repro` 原文落盘
`test_cli_repro_export`	CLI `--repro` 端到端：写出的凭证含矩阵形状、k 个特征值与 schema 标记

三个内核的数值结果都与稠密 LAPACK 参考比对，并覆盖了重复特征值与谱内目标。`python/README.md` 已重写为中文并同步了测试计数与行为描述（含质数布局回退与非有限值清洗）。需要注意"用屏蔽 SciPy 导入的方式验证降级后端"没有对应的独立实现——浏览器后端由 Pyodide 随 vendored numpy 与 scipy 一起覆盖，若要在 CPython 中强制复现纯 NumPy 降级路径需另写一个屏蔽 scipy 导入的脚本。

11.2 前端单元测试

`tests/emEigensolver.test.ts` 共 11 组、37 项用例，不启动 Worker，只覆盖纯逻辑与状态机：清单与注册表解析、`parseSigma` 的空值与非法输入处理、谱域与像素映射的边界行为、3D 色标（钳制、NaN 安全、零点为白、两臂可分）、极值定位（保留符号、单侧为空）、曲面缓冲（顶点数、中心化、高度缩放、颜色一致性）、3D 视图状态机（字符串参数、越界忽略、无结果时的 2D 回退不抛异常）、参数处理（数值下拉的白名单语义、非支持格式的文件被拒、`.mtx` 被接受并切换来源、合并后的数据来源下拉、刷新后 `source: 'file'` 的回落）、Figure Studio 面板构造（列排布、标签、字段序列、标题含 λ）、诊断报告构造（全部章节内联 SVG、零脚本零外部资源、对抗性矩阵名转义、中英双语、空轨迹省略、逐特征值谱点与发散色标、两个导出按钮在面板暴露）。

11.3 最小复现环境与命令

项	要求
Python	3.10+（浏览器侧由 Pyodide 提供的 CPython 承担）
必装依赖	`numpy >= 1.24`（BSD-3-Clause）
可选依赖	`scipy >= 1.10`（BSD-3-Clause），缺失即运行纯 NumPy 后端

项	要求
运行方式	独立：`python -m em_eigensolver.cli ...`；插件：在应用内选择样例或导入矩阵后点"运行求解"
确定性	所有随机初始化都走 `np.random.default_rng(seed)`；`seed` 为界面参数与配置字段，同配置同种子结果可复现
结果归档	特征值与特征向量写入 `.npz`（`numpy.load` 直接可读）；报告中的诊断含迭代数、matvec 数、位移轨迹与内存估算
运行特征	`main()` 按字母序执行全部 26 项、不提供筛选参数。其中 Jacobi-Davidson 与大规模样例相关的用例是主要耗时项，完整套件在 CPython 下为分钟到十数分钟量级；调试单项时可用 `pytest -k <用例名>`，或临时只保留目标用例。`stdout` 被重定向到文件时 `[PASS]` 行会因缓冲延迟出现，实时进度以 `stderr` 的内核日志为准

11.4 可复现凭证（`ergalics.em-repro`）与诊断报告

凭证内容。每次求解（CLI `--repro PATH`、插件"导出复现凭证"、Worker 报告自带的 `repro` 字段）产出一份自包含 JSON 凭证，schema 为 `ergalics.em-repro` v1：

字段	内容	复现用法
`matrix`	表示形式（csr/dense）、shape、nnz、复数标记 + SHA-256 指纹（`indptr` 加宽为 int64、`indices`、数值加宽为 float64/complex128 逐字节哈希）	比对 `matrix.hash` 断言"同一矩阵"——整矩阵逐字节哈希，O(nnz)、十万阶毫秒级，不稠密化不采样
`params` / `params_hash`	解析后 `SolverConfig` 全字段（canonical JSON：排序键、无空白）+ 其摘要	任何参数漂移（含 seed）都会改变 `params_hash`
`seed`	随机初始化种子（原文）	同配置同种子 → 同起始块

字段	内容	复现用法
`code`	包版本、Python/NumPy 版本、后端 + 全部内核模块的逐文件源码摘要与聚合值	任一内核被改动，`code.aggregate` 随之改变——凭证不会"假装"还是旧代码算的
`result`	方法、收敛、迭代数、matvec 数 + 特征值列表（全精度）、`eigenvalues_hash`（数值字节哈希）、最大残差	重跑后断言摘要（或逐位比对特征值列表）

实现位于 `repro.py`（纯 NumPy/stdlib，不依赖 scipy），因此同一份代码在 CPython 与 Pyodide 上产出同构凭证；指纹只做加宽（int32→int64、float32→float64）以消除后端索引/数值 dtype 差异，不改变数值本身。局限如实说明：凭证证明"同输入同代码同种子 → 同输出摘要"，不承诺跨 BLAS 实现的逐位一致（不同 BLAS 的内积顺序可能使特征值在 1e-14 量级上浮动，此时 `eigenvalues_hash` 会不同，但最大残差认证仍有效）。

诊断报告。插件"导出诊断报告"生成 `ergalics.em-diag-report` v1 自包含 HTML：求解参数与矩阵信息表、特征值谱条带图（σ 位置虚线标记）、逐特征对相对残差条形图（对数轴 + 容差线，超容差条目黄色警示）、外层收敛轨迹（每轮最大残差，对数轴）与全部模式场热图（发散色标）。全部图形为内联 SVG，零脚本零外部资源——适合作为附件提交的"工程诊断输出"，也与 PRD REQ-G 的验收口径一致。

11.5 一键复现

本插件另配套一枚独立的一键复现脚本，把"环境 → 测试 → 再生基准 → 核对"串成单条命令：

```
# 完整保真（含十万阶全量求解验证；慢机器用 --fast）
node scripts/repro-plugins.mjs --only=em

# 快速模式：跳过十万阶全量验证
node scripts/repro-plugins.mjs --only=em --fast
```

脚本对插件按序执行：`env`（定位可用 python、读取 numpy / scipy 版本）→ `tests`（运行 11.1 节的 `python/tests/test_all.py` 套件）→ `gen`（经同一入口 `solver.py` 在临时目录重新生成基准 JSON）→ `verify`（逐字段对比新 JSON 与提交基准 `bench/em-eigensolver-validate.json`）。退出码 `0` = 测试通过且确定性科学字段一致；`1` = 测试失败或存在确定性科学字段漂移。

手动复现亦可分步执行：

```
cd src/plugins/builtin/em-eigensolver/python

python -m em_eigensolver.cli --sample cavity_large --k 6 --out ./large.npz   # 样例矩阵单次求解（推荐极
python tests/test_all.py                                                      # 期望 AL
python benchmarks/validate_correctness.py --json ./em-validate.json           # 校验报告
python benchmarks/perf_bench.py --json ./em-perf.json                         # 性能曲线
```

说明：`validate_correctness.py` 会经 `--json` 把原始报告写到指定路径，并把 stress 工件写入 `bench/em-eigensolver-stress.json`（该工件用于归档对抗例证据）。`--fast` 等价于跳过 `validate_correctness.py` 内的 1e5 用例；`perf_bench.py` 对应 `--skip-large`。以上命令把报告写到当前目录便于与提交基准核对；`bench/` 下的提交态文件由脚本的临时目录重生成与其比对，不直接覆盖。

11.6 判定口径

脚本内置确定性字段比较器，把基准 JSON 字段按四类判定：

类别	判定方式	示例
精确断言	字符串 / 布尔必须完全相等	`converged`、`status`、`all_pass`
确定性数值	相对误差 ≤ 1e-6	`max_eigenvalue_error` 相关项
收敛量级（绝对容差）	绝对差 ≤ 1e-6，超限才 FAIL	`max_residual`、`eigenvalues`、`reference_eigenvalues`
机器相关	忽略或软容差（note 不判 FAIL）	`seconds`、`peak_memory_mb`、`iterations`、`matvecs`

- `max_residual` / `eigenvalues` 用绝对容差：这些收敛量在病态样例上落在 1e-9~1e-14（远低于 1e-8 认证容差），相对误差会被放大成数百%；以绝对差 ≤ 1e-6 判等，真正劣化的残差（超容差）仍触发 FAIL。求解收敛由精确的 `converged` / `status` / `max_eigenvalue_error` 断言独立守护。
- 对抗用例（`stress` 数组）内本征值差异归为 note：病态缩放（1e-9~1e10）或近简并谱下 JD 可能锁到相邻本征值，本征值本身属尘埃级浮点噪声，不构成复现断言；对应认证由上述精确断言守护。确定性 `reference` 数组保持严格绝对容差检查。
- 长度不一的数组（`reference` / `stress`）按语义键对齐：元素带 `case` 名的以 `case@method@sigma` 复合身份配对，不按下标硬比。因此 `--fast`（跳过十万阶行）只比较实际生成的用例，不会因缺行误报。
- 软项（note）不计入 FAIL；仅精确断言与确定性数值违背机组失败。

11.7 输出解读

`--only=em --fast` 成功输出示例：

```
=== em-eigensolver (eigen) reproduction: env → tests → regenerate → verify ===
python 3.10.11 | numpy 2.2.6 | scipy 1.15.3

[em-eigensolver]
  ok   em-eigensolver: standalone test suite passed
  ok   em-eigensolver: fresh bench matches committed baseline on all deterministic scientific fields

==========================================
RESULT: PASS — em-eigensolver reproduces and match committed baselines
==========================================
```

- `ok ... matches committed baseline`：新算结果与提交基准在全部确定性科学字段上一致。
- `FAIL` 行按以下顺序排查：① 测试套件是否通过；② 基准文件是否存在；③ 重生成是否报错；④ 哪些科学字段偏离（脚本打印字段路径与新旧值）。

11.8 故障排查

现象	处置
python discovery 失败	安装 Python 3.10+，或设环境变量 `PYTHON=/path/to/python`
numpy `<none>`	`pip install numpy`
scipy `<none>`	可选；内核自动用纯 NumPy CSR 后端，结果一致
em 全量验证太慢	加 `--fast`（跳过 1e5 用例）；或 `perf_bench.py --skip-large`
某科学字段 FAIL	按 11.7 节顺序排查

11.9 交付物对照

交付项	位置
一键复现脚本	`scripts/repro-plugins.mjs`（`--only=em`）
数值内核 + 验证	`src/plugins/builtin/em-eigensolver/python/`
解题说明书	`src/plugins/builtin/em-eigensolver/SOLUTION.md`
复现基准	`bench/em-eigensolver-validate.json`（校验）、`bench/em-eigensolver-results.json`（性能）、`bench/em-eigensolver-stress.json`（对抗例）

十二 安全与合规设计

关注点	设计
禁止硬编码	所有阈值（`tol`、`basis_dim`、`dense_threshold`、`near_singular_threshold`、`minres_rtol/maxiter`、块宽、种子）都是 `SolverConfig` 字段或内核参数；样例矩阵由带显式维度的构造器生成，没有针对任何输入的特殊分支
不读取参考结果	代码中不含任何参考答案、期望特征值表或外部结果文件的读取路径；收敛判定完全来自运行时计算的真实残差
不为公开用例调参	内核中不存在以输入名称或规模为条件的参数分支
不得稠密化	见第五节；稠密路径受 `dense_threshold` 严格限制，输入侧的稠密通道受 `max_dense_cells` 护栏限制，超限抛错
计算位置与数据边界	矩阵只在本地进程内解析与计算：浏览器中位于独立 Worker 的解释器文件系统内，导出经由本地文件下载；没有任何网络上传路径
进程隔离与复位	求解在专用 Worker 中执行，终止即整体重建；输入尺寸护栏在解析前生效，避免恶意大文件导致内存耗尽
依赖授权	numpy / SciPy（BSD-3-Clause）、Pyodide（MPL-2.0）、Three.js（MIT）；本插件与 Python 包均为 MIT
输入解析注意点	`.npy` 读取显式关闭 pickle（`allow_pickle=False`）；`.npz` 为兼容 SciPy 稀疏归档与稠密归档使用了 `allow_pickle=True`，只应加载可信来源的归档
脱敏与访问控制	插件不采集、不回传任何矩阵内容或元数据；参数与结果仅存在于本地会话与用户主动导出的文件中

十三 与赛题要求的对应

赛题《面向电磁谐振与微波器件仿真的高效、稳定求解方案》（AI+工业软件赛道）要求一套可复现、低内存、高精度的大规模非正定厄密稀疏矩阵特征值求解方案。下表把交付要求逐条映射到实现位置。

赛题要求	本方案的对应实现
a. 统一的矩阵读取、预处理、核心迭代、收敛控制与结果输出	`io_matrix.py`（读取与预处理）、`solver.py`（路由与收敛控制）、三个内核、`write_eigen_npz` 与 `driver.py`（输出）；CLI 与插件共用同一门面
b. 支持 NumPy / SciPy 格式输入，高效稀疏存储，按需恢复三角/完整矩阵	`.npy` / `.npz`（SciPy 稀疏与稠密两种载荷）/ `.mtx`（general、symmetric、hermitian、skew-symmetric、pattern、array）；本地 CPython 另可直读 `.h5` / `.hdf5`（h5py）、`.fits`（astropy）、`.nc`（SciPy NetCDF），缺包时抛出指名缺失包的 `MatrixIOError`；`csr_from_coo(hermitian_fill=True)` 与 `materialize_triangle(part='lower' / 'upper')` 完成三角恢复，全程不经过稠密
c. 适配非正定/不定厄密矩阵的 Krylov 或现代迭代算法	厚重启 Lanczos（Krylov-Schur）、块 LOBPCG、Jacobi-Davidson 三内核；核组件独立成文件，可单独导入
d. 位移策略与自适应位移	`σ` 驱动的位移逆变换（`λ = σ + 1/θ`）+ `lanczos_solve` 的自适应位移（内层迭代速率触发、固定方向步长倍增、轨迹记录）；JD 通过投影校正方程规避近奇异
e. 收敛性控制与精度	全部内核以真实残差判据收敛，结束后重新认证；`diagnostics` 报告迭代数、matvec 数、内层次数、位移轨迹与残差估计
f. 资源与性能优化，控制内存峰值，避免稠密化	`basis_dim` 作为内存旋钮、`memory_hint_mb` 估算、CSR 存储估算、`dense_threshold` 与 `max_dense_cells` 双重护栏；BLAS 多线程 + 纯 NumPy 后端行块多线程 matvec（阈值路由，见 9.4 节）；WebGPU SpMV 内核为平台级资产（浏览器内同步求解循环的委派边界如实标注，见第六节）
方案说明文档	本文档（算法原理、复杂度、预处理取舍、收敛性、并行现状、样例与基准、故障场景、安全设计）
可运行原型	`python -m em_eigensolver.cli`（输入 `.npy/.npz/.mtx`，输出 `.npz`）与内置插件的图形界面；两条路径共用同一求解核心
核心算法组件与开源依赖说明	内核文件按算法独立；依赖与授权见第十二节；调用方式见 `python/README.md` 与第九节的命令示例
样例数据与基准、默认配置样例	5 个参数化样例 + 1 个随仓库分发的 `.mtx` 示例；`config.example.json` 给出完整默认配置

赛题要求	本方案的对应实现
故障场景与验证	见第十节；测试覆盖重特征值、谱内目标、不定系统与降采样边界
可复现材料	源码、`requirements.txt`、`python/README.md`（中文）、26 项 Python 测试与 35 项前端用例、`benchmarks/`（性能与正确性基准脚本 + 实测结果 JSON）、同种子可复现的随机初始化、可复现凭证 `repro.json`（CLI `--repro` / 插件一键导出，见 11.4 节）
不得将不可验证的闭源服务作为核心能力	核心算法为纯 Python + NumPy，无闭源依赖，无网络调用
须实现基本的敏感信息保护	见第十二节：无上传、无采集，导出仅本地文件，加载不可信 `.npz` 的风险已注明

按评审维度看，本方案的证据分布为：任务定义与通用性对应第一、十三节；算法正确性与精度对应第四、十一节（真实残差认证 + LAPACK 参考比对）；性能与资源效率对应第五、九节（`n = 102400` 样例、内存估算、不得稠密化的实现约束）；稳定性与鲁棒性对应第三、十节（近奇异位移、零附近密集谱、重特征值各有专门机制与样例）；创新性与工程落地对应第二、七节（不精确位移逆、双判据收缩、自适应位移，以及浏览器内运行同一套求解器的工程形态）；可复现性与文档对应本文档与第九、十一节。

十四 已知边界与后续工作

项	现状	影响与建议方向
广义特征问题	仅支持 `A x = λ x`	电磁问题的常见形式是 `A x = λB x`（`B` 为质量矩阵）。目前需使用者先做变换；后续可在门面内接受 `B` 并对内层系统改用 `B` 内积
内层预条件	默认不施加，仅 LOBPCG 保留钩子	无预条件时谱内密集目标的迭代次数偏高；可考虑对可判定为正定的位移区间启用位移相关的 SPD 预条件
并行与硬件加速	BLAS 多线程 + 纯 NumPy 后端行块多线程 matvec（线程池仅在 $n \gtrsim 4\times10^4$ 启用，实测拐点见 9.4 节）；无 MPI；GPU SpMV 内核已实现并暴露为平台级独立 API	浏览器内同步求解循环无法逐 matvec 委派异步 GPU，求解主循环保持 CPU f64 精确路径（见第六节）——十万阶以上或大批量求解时性能仍受单线程 WASM 限制；同步宿主可经 `csr.set_gpu_spmv` 桥接入

项	现状	影响与建议方向
性能与扩展路线图（Roadmap）	当前为 CPU f64 主循环 + BLAS/行块多线程；GPU SpMV 计算着色器内核已具备但未进入同步热循环	演进顺序：① **混精度 GPU 加速**——把 `csr` 里已实现的 WebGPU f32 SpMV 挂在同步宿主桥（`csr.set_gpu_spmv`）上做近似乘，配合 CPU f64 数学件残差复核，在保持认证精度的同时摊薄十万阶 matvec 成本；② **原生 sidecar**——对大批量谱扫/更大矩阵，经 WASIX 或原生 Node 扩展把稀疏路径放到多核原生 CPython/OpenMP 上，绕开 WASM 单线程天花板；③ **MPI 分布式**——内核已与后端解耦，块 Krylov 可按进程分片平移，供集群大规模谱扫描；④ **位移相关 SPD 预条件**——在可判定为正定的位移区间内层启用预条件，压低密集谱目标的迭代次数
大规模内点位移逆的精度上限与自适应位移逃逸	`n = 102400`、`σ = 0.5` 时谱内目标附近过密，MINRES 内层在巨大不定系统上持续高负载，触发自适应位移把 σ 逐步移出谱（n = 4096 诊断实测 `shift_history`：0.5 → 0.815 → … → 10.277，越出谱顶；1e5 阶对照运行返回的特征值即谱顶 8.0424，见 9.4 节），外层收敛到的实为距终态 σ 最近的谱端认证特征对而非 σ 邻域目标，真实残差约 `5.9e-5`（同一矩阵极端路径 `3.4e-8`）	密集谱内点的 σ 邻域锁定对位移逆不可达——这正是 `auto` 路由把谱内目标交给 JD 的原因（JD 投影校正天然规避近奇异）；显式位移逆仅建议用于谱间隙目标，内点目标请改用 JD，未来可为可判定为正定的位移区间引入内层预条件
谱顶/谱底密集簇的 JD 锁定速度	`cavity_small`（900 阶）`σ = 8.3` 求近顶部 6 个模式时，400 步只能锁定 5 个（k=4 可在 284 步收敛，见 4.4 基准）	顶部密集簇叠加简并时收缩进度慢属已知行为；建议减小 k 或加大 `max_cycles`
前端端到端覆盖	本插件只有纯逻辑单测，无 e2e 套件	求解链路（含 Pyodide 引导）没有自动化回归；建议补一条点击样例并断言出现收敛状态的端到端用例
极端类型选择	插件固定 `which = 'LM'`	界面无法直接求最大/最小代数特征值；可在参数面板补一个下拉（Python 包与 CLI 层已支持 `LM` / `LA` / `SA`）

ERGALICS STUDIO · 宗旨

让每一次科学计算，

都发生在离思想最近的地方

Ergalics Studio 把复杂度收进浏览器：安装、配置与数据搬运都不该出现在科研的路上。打开即可计算，数据留在本机，四种工作模式共享同一份数据——一张图、一条流程与一行代码，都是同一次工作的不同表达。

工具越安静，思想就越清楚。我们做的每一处取舍，都为了把注意力还给问题本身。

作者 SnowLeopard-io github.com/SnowLeopard-io

官网 snowleopard-io.github.io/ErgalicsStudio
GitHub github.com/SnowLeopard-io/ErgalicsStudio
Gitee gitee.com/cnt-code/ergalics-studio

独立专题文档 · Standalone Note